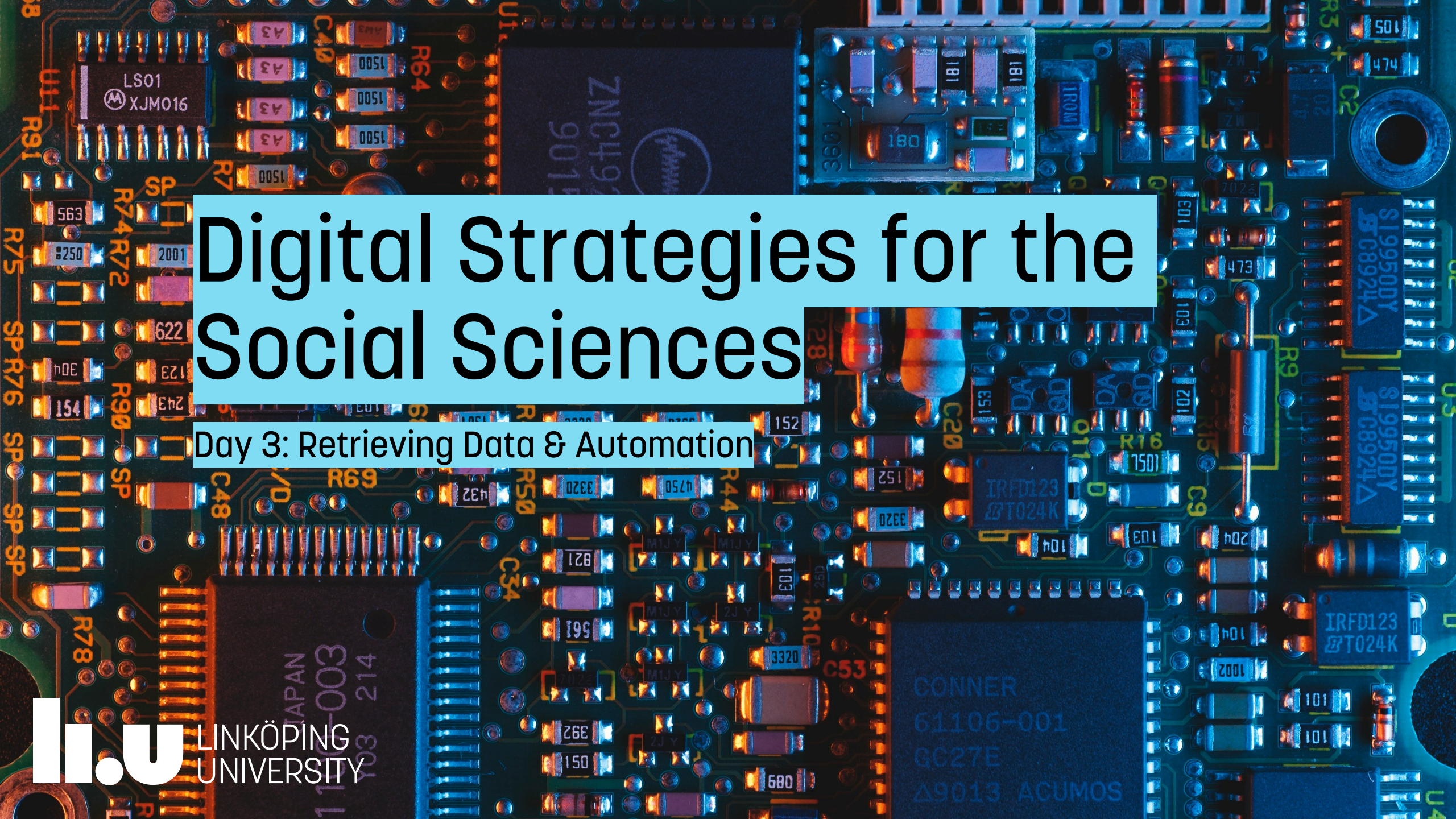




Digital Strategies for the Social Sciences

Day 3: Retrieving Data & Automation

Survey Results

- Everything seems well! That's great!
- (Also, I will continue to try to speak slowly!)

A Story from Yesterday

A real-life example of the GDPR and the ethical principle of “Respect for Persons”

Today's Agenda

- We know how requesting websites work
- We know how to save them to disk
- Now it's about actually getting the data out of the HTML

Reminder

The goal is to take HTML and turn it into a DataFrame

Table of AMD processors

Article [Talk](#) [Read](#) [Edit](#) [View history](#) [Tools](#)

From Wikipedia, the free encyclopedia

Architecture	Fabrication (nm)	Family	Release Date	Code name	Model Group	Cores	SMT	Clock rate (MHz)	Bus Speed & Type ^[a]	Cache	Socket	Memory Controller	SIMD
										L1	L2	L3	
Am386		Am386		Sx/SxL/SxLV ^[1]		1	No	25–40 ^[1]	FSB		100 PQFP ^[1]	discrete	
Am486 ^[2]	500, 350	Am486				1	No	25–120	FSB	8	168 pin PGA 208 SQFP	discrete	
	500, 350	Enhanced Am486						66–120	FSB	8, 8/16	168 pin PGA 208 SQFP ^[3]		
Am5x86	350	Am5x86		X5-133		1	No	133	33 FSB	16	Socket 3 Socket 9		
K5	500, 350	AMD K5		SSA/5, 5k86		1	No	75–133	50, 60, 66 FSB	8+16	0		
K6	350, 250	AMD K6		Model 6, Littlefoot		1	No	166–300	50, 60, 66 FSB	32+32	0	Socket 7	MMX
	250, 180	AMD K6-2		Chomper, Chomper Extended, mobile				166–550	66, 95, 97, 100 FSB	32+32	0, 128	Super Socket 7	discrete
	250, 180	K6-3		Sharptooth				400–550	66, 95, 96.2, 100 FSB	32+32	256	Super Socket 7	MMX, 3DNow!
	250	Athlon		Argon				500–700			512	Slot A	MMX, 3DNow!
				Pluto/Orion				550–1000	100 FSB			Slot A Socket A	
	180			Thunderbird				1000–1400					
	180	Athlon XP/MP		Palomino	1500+ – 2100+			1333–1733	133 FSB				
				Thoroughbred	1600+ – 2800+			1400–2250					
	130			Thorton	2000+ – 3100+			1667–2200	133, 166, 200 FSB				

	Architecture str	Fabrication(nm) str	Family str	Release Date str	Code name str
1	Architecture	Fabrication(nm)	Family	Release Date	Code name
2	Am386	<empty>	Am386	<empty>	Sx/SxL/SxLV[1]
3	Am486[2]	500, 350	Am486	<empty>	<empty>
4	Am486[2]	500, 350	Enhanced Am486	<empty>	<empty>
5	Am5x86	350	Am5x86	<empty>	X5-133
6	K5	500, 350	AMD K5	<empty>	SSA/5, 5k86
7	K6	350, 250	AMD K6	<empty>	Model 6, Littlefoot
8	K6	250, 180	AMD K6-2	<empty>	Chomper, Chomper Ext...
9	K6	250, 180	K6-3	<empty>	Sharptooth
10	K7	250	Athlon	<empty>	Argon
11	K7	180	Athlon	<empty>	Pluto/Orion
		180	Athlon	<empty>	Thunderbird
		180	Athlon XP/MP	<empty>	Palomino
		130	Athlon XP/MP	<empty>	Thoroughbred
15	K7	130	Athlon XP/MP	<empty>	Thorton
16	K7	130	Athlon XP/MP	<empty>	Barton
17	K7	180	Duron	<empty>	Spitfire
18	K7	180	Duron	<empty>	Morgan
19	K7	130	Duron	<empty>	Applebred
20	K7	180	Athlon 4	<empty>	Corvette
21	K7	130	Athlon XP-M	<empty>	Thoroughbred
22	K7	130	Athlon XP-M	<empty>	Barton
23	K7	180	Mobile Duron	<empty>	Spitfire
24	K7	180	Mobile Duron	<empty>	Camaro
25	K7	130	Sempron	<empty>	Thoroughbred-B
26	K7	130	Sempron	<empty>	Thorton
27	K7	130	Sempron	<empty>	Barton
28	K8	130	Duron	<empty>	Sledgehammer

Reminder

The goal is to take HTML and turn it into a DataFrame

Table of AMD processors

Article Talk

Read Edit View history Tools

From Wikipedia, the free encyclopedia

Architecture	Fabrication (nm)	Family	Release Date	Code name	Model Group	Cores	SMT	Clock rate (MHz)	Bus Speed & Type ^[a]	Cache	Socket	Memory Controller	SIMD
										L1	L2	L3	
Am386		Am386		Sx/SxL/SxLV ^[1]		1	No	25–40 ^[1]	FSB			100 PQFP ^[1]	discrete
Am486 ^[2]	500, 350	Am486				1	No	25–120	FSB	8		168 pin PGA 208	
	500, 350	Enhanced Am486						66–120					
Am5x86	350	Am5x86		X5-133		1	No	133					
K5	500, 350	AMD K5		SSA/5, 5k86		1	No	75–133	50, 66, 100 FSB				
K6	350, 250	AMD K6		Model 6, Littlefoot		1	No	166–300	50, 66, 100 FSB				
	250, 180	AMD K6-2		Chomper, Chomper Extended, mobile			No	166–550	66, 95, 97, 100 FSB	32+32	0, 128		Super Socket 7
	250, 180	K6-3		Sharptooth				400–550	66, 95, 96.2, 100 FSB	32+32	256	512, 1024	Super Socket 7
	250	Athlon		Argon				500–700					
				Pluto/Orion				550–1000	100 FSB				
	180			Thunderbird				1000–1400				Slot A Socket A	
	180	Athlon XP/MP		Palomino	1500+ – 2100+			1333–1733	133 FSB				
	130			Thoroughbred	1600+ – 2800+			1400–2250					
				Thorton	2000+ – 3100+			1667–2200	133, 166, 200 FSB				

```
1 library(httr)
2 library(XML)
3
4 url <- "https://en.wikipedia.org/wiki/Table_of_AMD_processors"
5 response <- GET(url)
6 html <- content(response, as = "text")
7
8 dom <- htmlParse(html)
9 cpuNames <- xpathApply(dom, "//table/tbody/tr/td[2]", xmlValue)
10
```

	Architecture str	Fabrication(nm) str	Family str	Release Date str	Code name str
1	Architecture	Fabrication(nm)	Family	Release Date	Code name
2	Am386	<empty>	Am386	<empty>	Sx/SxL/SxLV[1]
3	Am486[2]	500, 350	Am486	<empty>	<empty>
4	Am486[2]	500, 350	Enhanced Am486	<empty>	<empty>
5	Am5x86	350	Am5x86	<empty>	X5-133
			AMD K5	<empty>	SSA/5, 5k86
			AMD K6	<empty>	Model 6, Littlefoot
			AMD K6-2	<empty>	Chomper, Chomper Ext...
			K6-3	<empty>	Sharptooth
			Athlon	<empty>	Argon
			Athlon	<empty>	Pluto/Orion
			Athlon	<empty>	Thunderbird
			Athlon XP/MP	<empty>	Palomino
			Athlon XP/MP	<empty>	Thoroughbred
			Athlon XP/MP	<empty>	Thorton
			Athlon XP/MP	<empty>	Barton
			Duron	<empty>	Spitfire
			Duron	<empty>	Morgan
			Duron	<empty>	Applebred
			Athlon 4	<empty>	Corvette
			Athlon XP-M	<empty>	Thoroughbred
			Athlon XP-M	<empty>	Barton
			Mobile Duron	<empty>	Spitfire
			Mobile Duron	<empty>	Camaro
			Sempron	<empty>	Thoroughbred-B
			Sempron	<empty>	Thorton
			Sempron	<empty>	Barton
			Duron	<empty>	Sledgehammer

Thinking in Trees

I'm gonna plant a tree in your mind

Thinking in Trees

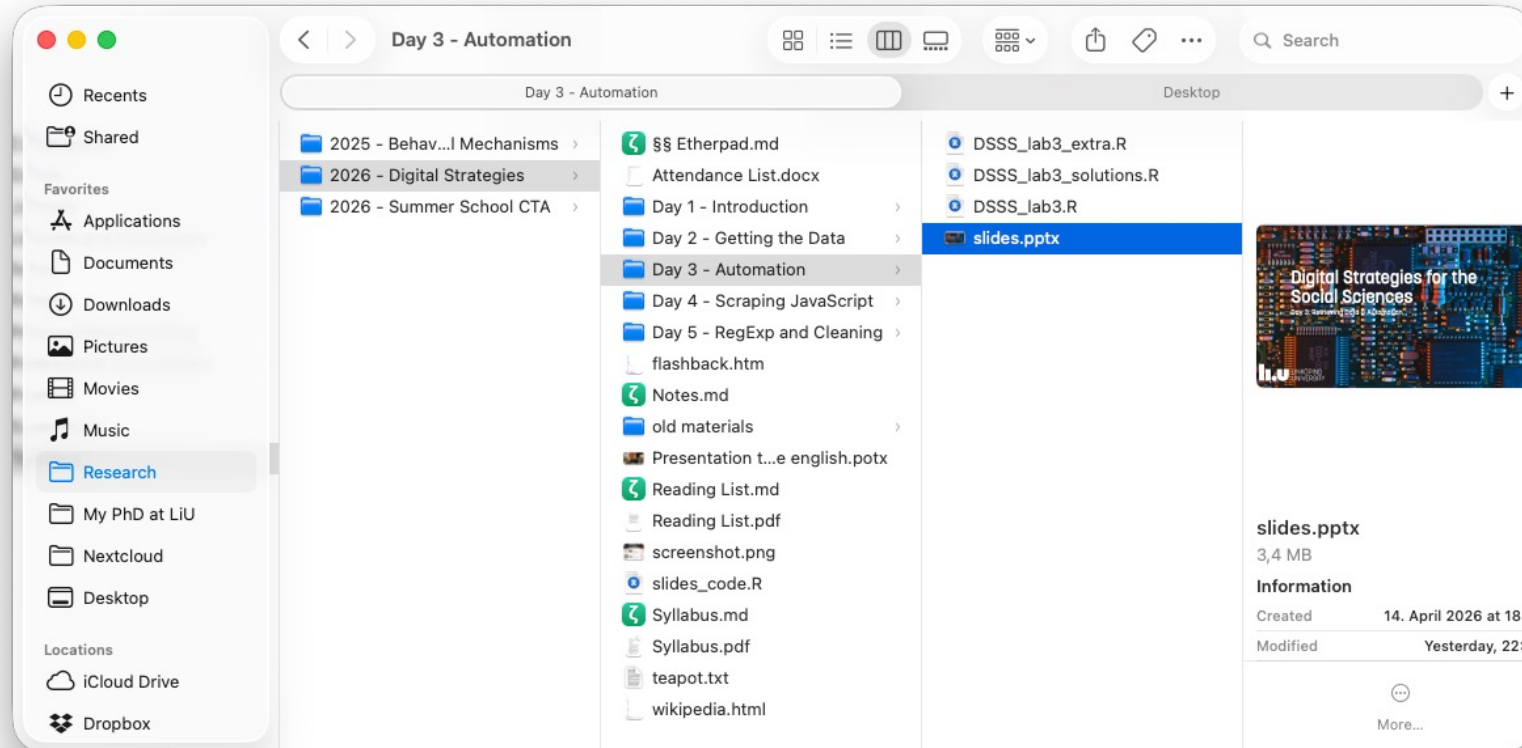
You all have a very visible tree on your computers that you are interacting with every day. Which one is it?

Thinking in Trees

You all have a very visible tree on your computers that you are interacting with every day. Which one is it?

Your file system

Thinking in Trees



HTML is a Tree Structure

```
<!DOCTYPE html>
<html>
  <head>
    <title>This is a title</title>
  </head>
  <body>
    <div>
      <p>Hello world!</p>
    </div>
  </body>
</html>
```

HTML is a Tree Structure

- Note on trees:
 - They grow from the bottom up in real life
 - In computer science, they are usually visualized either
 - top-to-bottom,
 - or left-to-right

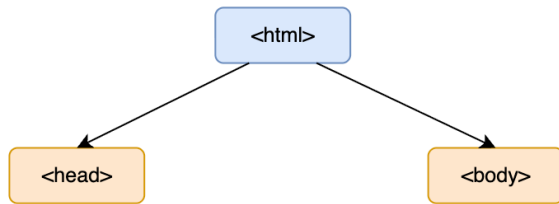
```
<!DOCTYPE html>
<html>
  <head>
    <title>This is a title</title>
  </head>
  <body>
    <div>
      <p>Hello world!</p>
    </div>
  </body>
</html>
```

HTML is a Tree Structure

<html>

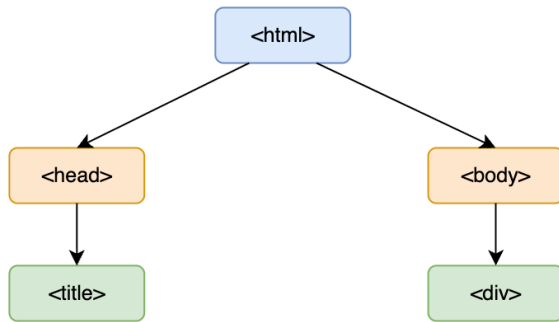
```
<!DOCTYPE html>
<html>
  <head>
    <title>This is a title</title>
  </head>
  <body>
    <div>
      <p>Hello world!</p>
    </div>
  </body>
</html>
```

HTML is a Tree Structure



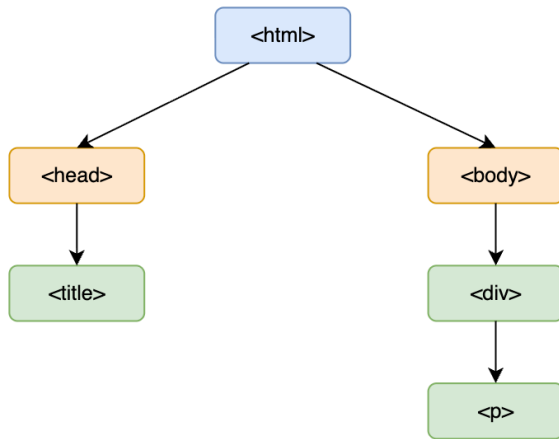
```
<!DOCTYPE html>
<html>
  <head>
    <title>This is a title</title>
  </head>
  <body>
    <div>
      <p>Hello world!</p>
    </div>
  </body>
</html>
```

HTML is a Tree Structure



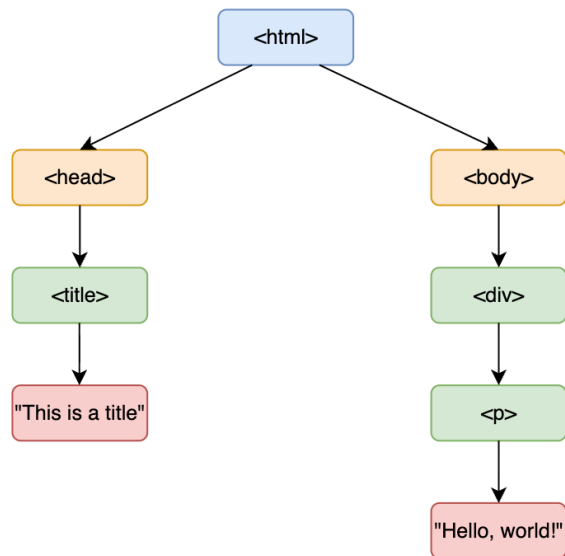
```
<!DOCTYPE html>
<html>
  <head>
    <title>This is a title</title>
  </head>
  <body>
    <div>
      <p>Hello world!</p>
    </div>
  </body>
</html>
```

HTML is a Tree Structure



```
<!DOCTYPE html>
<html>
  <head>
    <title>This is a title</title>
  </head>
  <body>
    <div>
      <p>Hello world!</p>
    </div>
  </body>
</html>
```

HTML is a Tree Structure



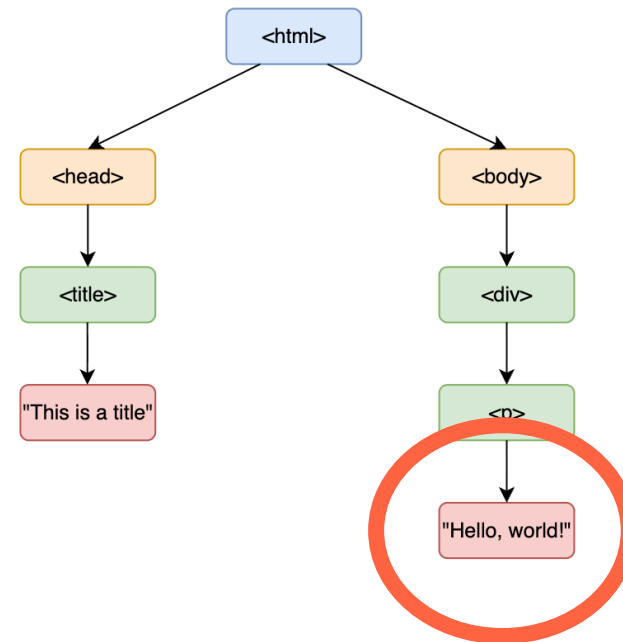
```
<!DOCTYPE html>
<html>
  <head>
    <title>This is a title</title>
  </head>
  <body>
    <div>
      <p>Hello world!</p>
    </div>
  </body>
</html>
```

Introducing XPath

Navigating Across Trees

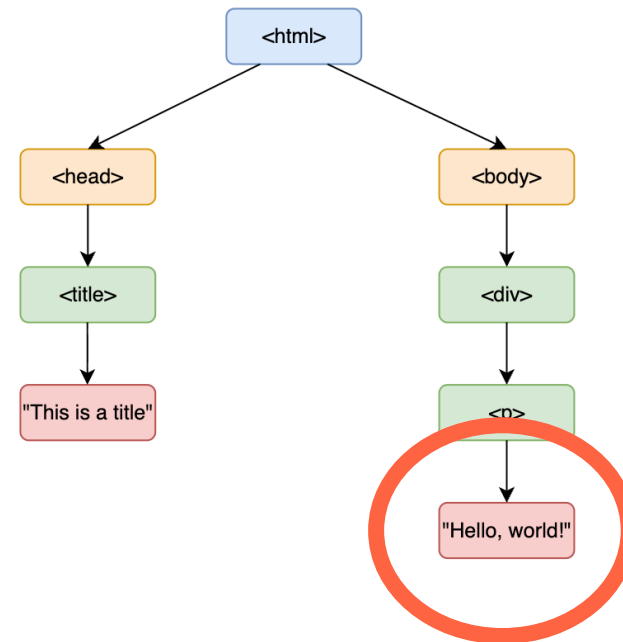
Trees are easy to navigate

- What if we want to know the value of all paragraphs in this document?



Trees are easy to navigate

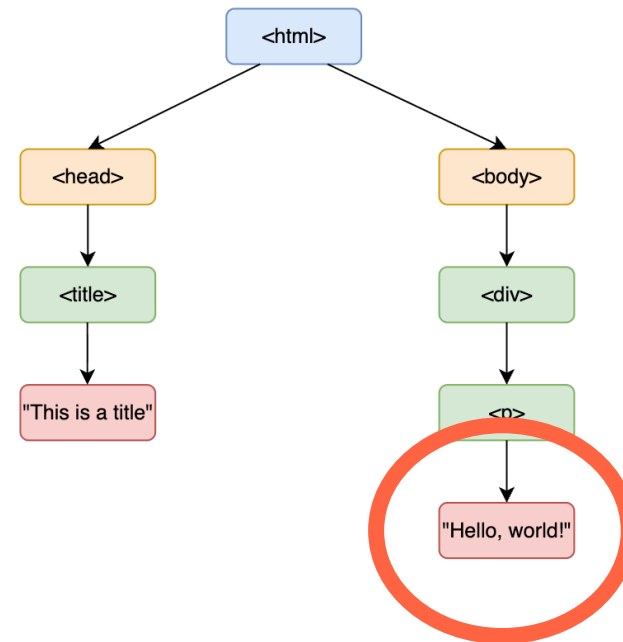
- What if we want to know the value of all paragraphs in this document?
- We have the HTML — but as a string



Trees are easy to navigate

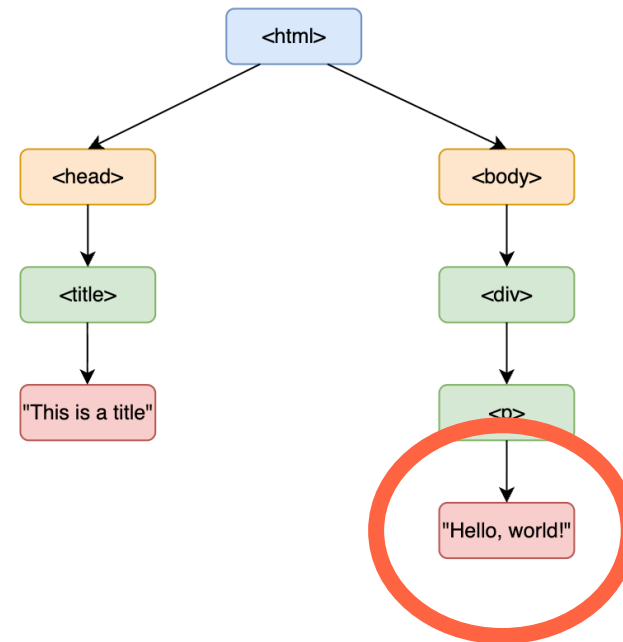
- What if we want to know the value of all paragraphs in this document?
- We have the HTML — but as a string
- Convert it to an actual (computing) tree:

```
url <- "https://en.wikipedia.org/wiki/Table_of_AMD_processors"  
response <- GET(url)  
html <- content(response, as = "text")  
  
dom <- htmlParse(html)
```



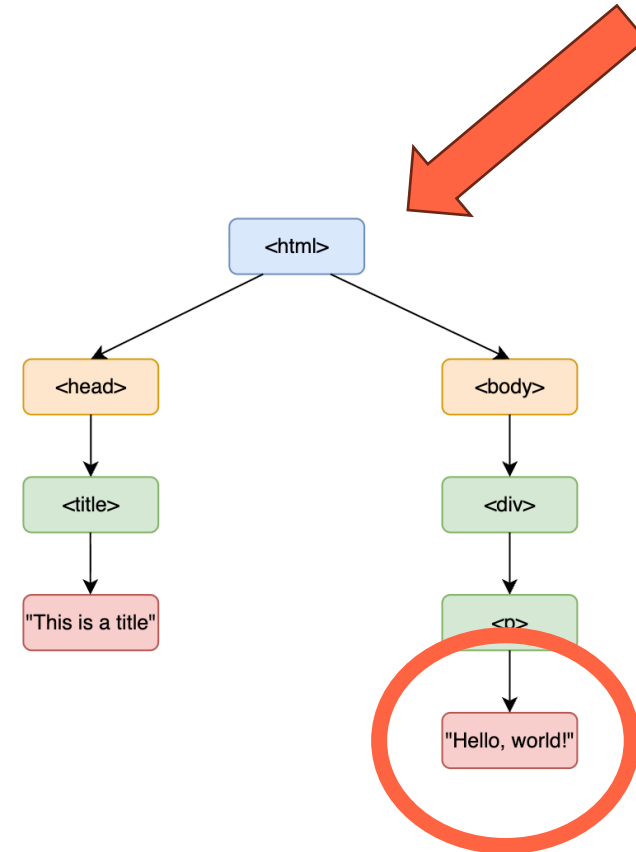
Trees are easy to navigate

- What if we want to know the value of all paragraphs in this document?
- We have the HTML — but as a string
- Next, we need to find the path to what we want:



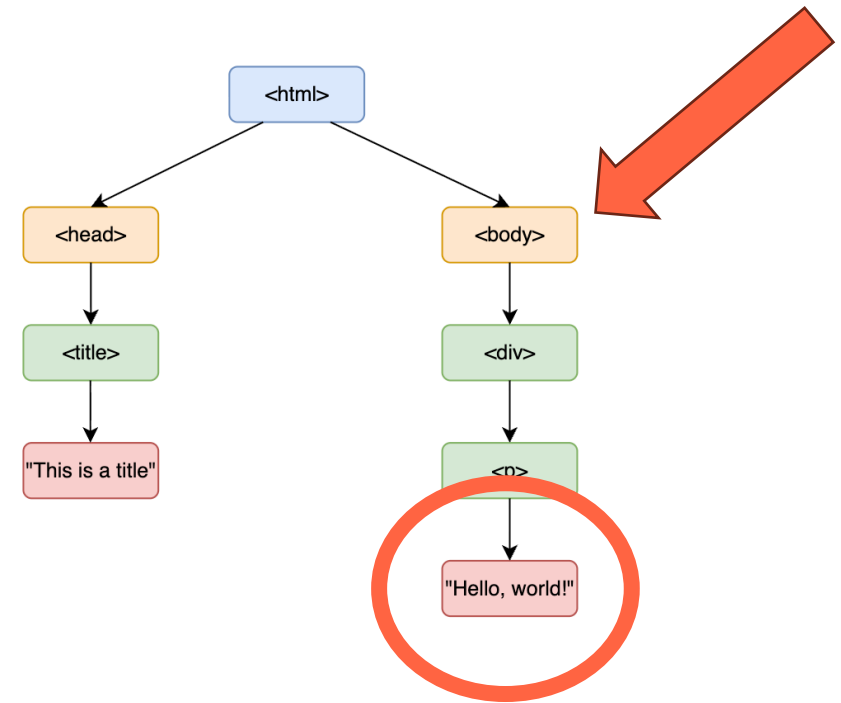
Trees are easy to navigate

- What if we want to know the value of all paragraphs in this document?
- We have the HTML — but as a string
- Next, we need to find the path to what we want:
 - `html`



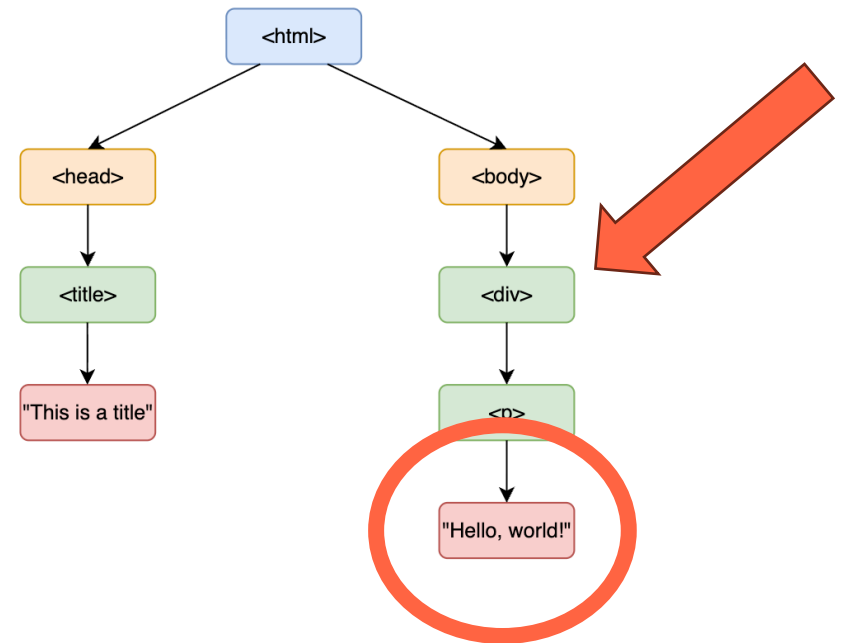
Trees are easy to navigate

- What if we want to know the value of all paragraphs in this document?
- We have the HTML — but as a string
- Next, we need to find the path to what we want:
 - `html` → `body`



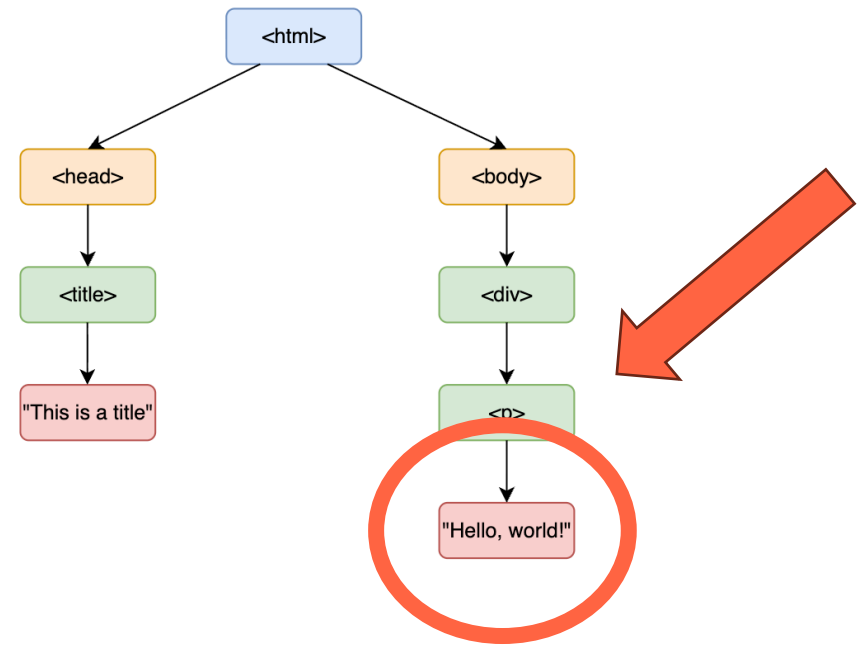
Trees are easy to navigate

- What if we want to know the value of all paragraphs in this document?
- We have the HTML — but as a string
- Next, we need to find the path to what we want:
 - `html` → `body` → `div`



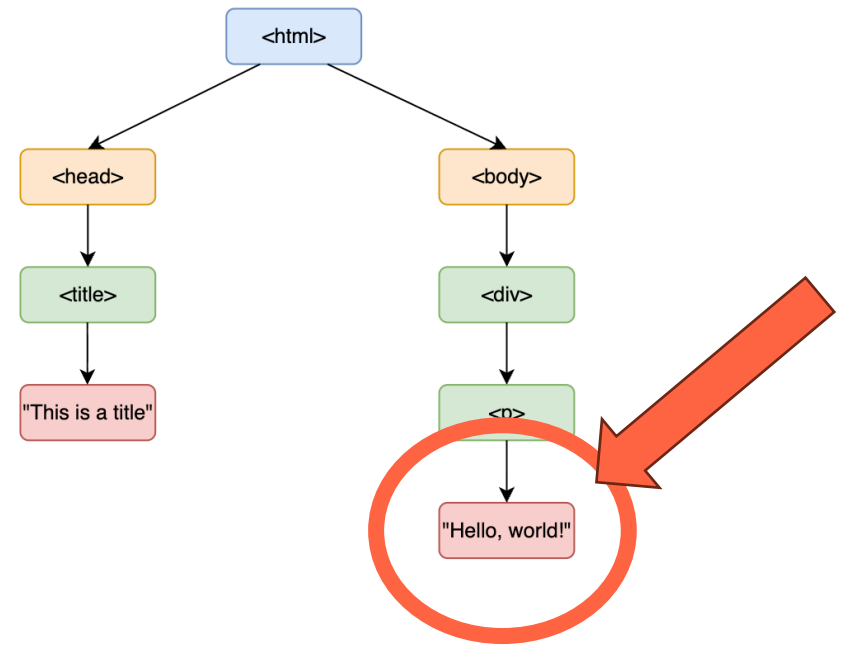
Trees are easy to navigate

- What if we want to know the value of all paragraphs in this document?
- We have the HTML — but as a string
- Next, we need to find the path to what we want:
 - `html` → `body` → `div` → `p`



Trees are easy to navigate

- What if we want to know the value of all paragraphs in this document?
- We have the HTML — but as a string
- Next, we need to find the path to what we want:
 - `html` → `body` → `div` → `p` → (value of node)



XPath

- XPath is a selection language that allows picking and choosing specific parts of an HTML tree

```
//table[@class="wikitable"]/tbody/tr/td[2]
```

XPath

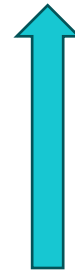
- XPath is a selection language that allows picking and choosing specific parts of an HTML tree

```
//table[@class="wikitable"]/tbody/tr/td[2]
```

XPath

- XPath is a selection language that allows picking and choosing specific parts of an HTML tree

`//table[@class="wikitable"]/tbody/tr/td[2]`



“Start from current node”

XPath

- XPath is a selection language that allows picking and choosing specific parts of an HTML tree

```
//table[@class="wikitable"]/tbody/tr/td[2]
```



“Find all tables that are contained within the current node”

XPath

- XPath is a selection language that allows picking and choosing specific parts of an HTML tree

```
//table[@class="wikitable"]/tbody/tr/td[2]
```



“Find all tables that are contained within the current node
...that have the attribute ‘class’ set to ‘wikitable’.”

XPath

- XPath is a selection language that allows picking and choosing specific parts of an HTML tree

```
//table[@class="wikitable"]/tbody/tr/td[2]
```



“Inside them, find all tbody-elements.” (there is only one)

XPath

- XPath is a selection language that allows picking and choosing specific parts of an HTML tree

```
//table[@class="wikitable"]/tbody/tr/td[2]
```

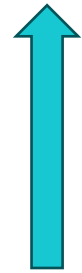


“Then, all tr elements” (tr = table row)

XPath

- XPath is a selection language that allows picking and choosing specific parts of an HTML tree

```
//table[@class="wikitable"]/tbody/tr/td[2]
```



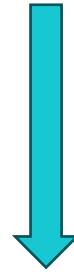
“Finally, select the second cell” (td = table data)

XPath

- XPath is a selection language that allows picking and choosing specific parts of an HTML tree

```
//table[@class="wikitable"]/tbody/tr/td[2]
```

Wrap this expression into `xpathSApply`



```
xpathSApply(  
  dom,  
  '//table[@class="wikitable"]/tbody/tr/td[2]',  
  xmlValue  
)
```

XPath

- Xpath is a selection language that allows picking and choosing specific parts of an HTML tree

```
//table[@class="wikitable"]/tbody/tr/td[2]
```

Then you get a list of that column!

```
> print(xpathSApply(dom, "//table/tbody/tr/td[2]", xmlValue))
[1] "Am386\n"
[3] "Enhanced Am486\n"
[5] "AMD K5\n"
[7] "AMD K6-2\n"
[9] "Athlon\n"
[11] "Thunderbird\n"
[13] "\n"
[15] "Barton\n"
[17] "Morgan\n"
[19] "Athlon 4\n"
[21] "Barton\n"
[23] "Camaro\n"
[25] "Thorton\n"
[27] "Opteron\n"
[29] "800\n"
[31] "Troy\n"
[33] "Denmark\n"
[35] "Egypt\n"
```

```
"Am486\n"
"Am5x86\n"
"AMD K6\n"
"K6-3\n"
"\n"
"Athlon XP/MP\n"
"Thorton\n"
"Duron\n"
"\n"
"Athlon XP-M\n"
"Mobile Duron\n"
"Sempron\n"
"Barton\n"
"200\n"
"\n"
"Athens\n"
"Italy\n"
"Santa Ana\n"
```

XPath

- Xpath is a selection language that allows picking and choosing specific parts of an HTML tree

```
//table[@class="wikitable"]/tbody/tr/td[2]
```

Then you get a list of that column!

(these are already
hints for why we
will need some cleaning)

```
> print(xpathApply(dom, "//table/tbody/tr/td[2]", xmlValue))  
[1] "Am386\n"  
[3] "Enhanced Am386\n"  
[5] "AMD K5\n"  
[7] "AMD K6-2\n"  
[9] "Athlon\n"  
[11] "Thunderbird\n"  
[13] "\n"  
[15] "Barton\n"  
[17] "Morgan\n"  
[19] "Athlon 4\n"  
[21] "Barton\n"  
[23] "Camaro\n"  
[25] "Thorton\n"  
[27] "Opteron\n"  
[29] "800\n"  
[31] "Troy\n"  
[33] "Denmark\n"  
[35] "Egypt\n"
```

```
"Am486\n"  
"Am5x86\n"  
"AMD K6\n"  
"K6-3\n"  
"\n"  
"Athlon XP/MP\n"  
"Thorton\n"  
"Duron\n"  
"Athlon XP-M\n"  
"Mobile Duron\n"  
"Sempron\n"  
"Barton\n"  
"Duron\n"  
"\n"  
"Athlon\n"  
"Duron\n"  
"Santa Ana\n"
```

Anatomy of xpathSApply

This is the path
to your elements



```
result <- xpathSApply(dom, "//table[@class='wikitable']/tr/td[2]", xmlValue)
```



This is an HTML tree
(not the original HTML string!)



This is what you want
from these elements:

xmlValue
xmlAttr

Anatomy of xpathSApply

Sidenote: "DOM" is short
for Document Object Model

This is the path
to your elements



```
result <- xpathSApply(dom, "//table[@class='wikitable']/tr/td[2]", xmlValue)
```



This is an HTML tree
(not the original HTML string!)



This is what you want
from these elements:

xmlValue
xmlAttr

Anatomy of xpathSApply

This is the path
to your elements



```
result <- xpathSApply(dom, "//table[@class='wikitable']/tr/td[2]", xmlValue)
```



This is an HTML tree
(not the original HTML string!)



This is what you want
from these elements:

xmlValue
xmlAttr

*Other Sidenote: xpathSApply is a variant
of R's built-in sapply ("simple apply")*

XPath Syntax

`section//div`

Any div that is somewhere within a section element

`/html/head/title`

Absolute path (from the root)

`./tr`

Relative path from current element

`./tr`

Also works with “//” (= any depth)

`div[@id=“some-id”]`

A div that has exactly the ID property “some-id”

`//table[contains(@class, 'wikitable')]`

A table whose class property contains “wikitable” somewhere

`//a[contains(text(), “sociology”)]`

Any a whose text contents contain “sociology”

`section/*/div`

Any div that is a child of any element that is a direct child of a section

Small Tree Glossary

- “Trees” in computing have a bunch of notation that can be helpful:
 - **Root**: The topmost node in a tree
 - **Branch**: A node that has a parent and children
 - **Leaf**: A node that has no children
 - **Parent**: Node containing the current one
 - **Child**: Node contained within the current one
 - **Sibling**: A node that has the same parent as the current one
- Also some useful properties:
 - Except the root node, every node has exactly one parent
 - A node can be empty
 - Multiple unrelated nodes can have the same XPath to them

Small Tree Glossary

- “Trees” in computing have a bunch of notation that can be helpful:
 - **Root**: The topmost node in a tree
 - **Branch**: A node that has a parent and children
 - **Leaf**: A node that has no children
 - **Parent**: Node containing the current one
 - **Child**: Node contained within the current one
 - **Sibling**: A node that has the same parent as the current one
- Also some useful properties:
 - Except the root node, every node has exactly one parent
 - A node can be empty
 - Multiple unrelated nodes can have the same XPath to them

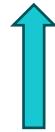
Sidenote: Such a tree is just a special form of a network, so some terms from the Social Network Analysis course also apply: degree, edges, etc.!

Navigating a Tree

```
result <- xpathSApply(dom, "//table[@class='wikitable']/tr/td[2]", xmlValue)
```

Navigating a Tree

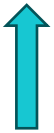
```
result <- xpathSApply(dom, "//table[@class='wikitable']/tr/td[2]", xmlValue)
```



You can leave this out!

Navigating a Tree

```
node <- xpathSApply(dom, "//table[@class='wikitable']/tr/td[2]")
```

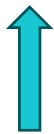


Now, `xpathSApply` neither returns attributes nor values, but actual nodes.

You can pass them to `xpathSApply` instead of `dom`!

Navigating a Tree

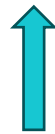
```
node <- xpathSApply(dom, "//table[@class='wikitable']/tr/td[2]")
```



Then, you can write relative XPaths
that start at these nodes instead

Navigating a Tree

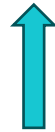
```
result <- xpathSApply(dom, "//table[@class='wikitable']/tr/td[2]", xmlValue)
```



Also, `xmlValue` and
`xmlAttrs` are not the only
available functions

Navigating a Tree

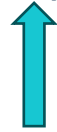
```
result <- xpathSApply(dom, "//table[@class='wikitable']/tr/td[2]", xmlValue)
```



Another mapping is
to return `xmlChildren`

Navigating a Tree

```
result <- xpathSapply(dom, "//table[@class='wikitable']/tr/td[2]", xmlValue)
```



Because this function is essentially
an `sapply`, but for trees.

Navigating a Tree

```
tds <- xpathSApply(dom, "//table[@class='wikitable']/tr", xmlChildren)
```

Navigating a Tree

```
tds <- xpathSApply(dom, "//table[@class='wikitable']/tr", xmlChildren)

for(td in tds) {
  print(xpathSApply(td, ".", xmlValue))
}
```

Navigating a Tree

```
tds <- xpathSApply(dom, "//table[@class='wikitable']/tr", xmlChildren)
```

```
for(td in tds) {  
  print(xpathSApply(td, ".", xmlValue))  
}
```

```
for(td in tds) {  
  print(xpathSApply(td, "..", xmlValue))  
}
```

Automation

Automate the boring stuff

Automation

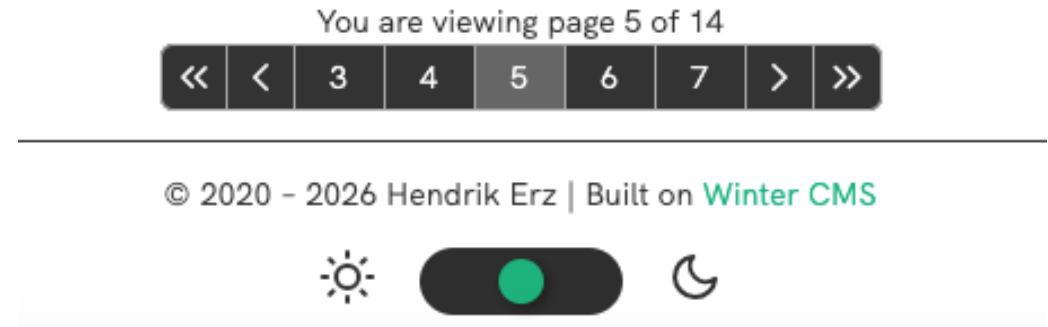
- Typically, you will have to scrape multiple things from multiple pages
- However, those will usually be patterned
- What works on one sub-page likely works on another page
- Doing all the scraping manually (in R) is tedious

Automation

- What to automate:
 - Pagination
 - Searches/Parameters
- How to automate:
 - for-loops
 - while-loops
 - Modular code (functions)

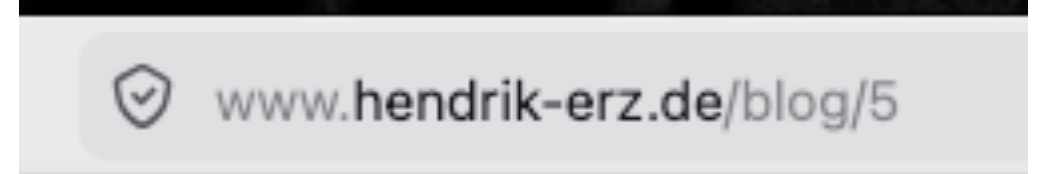
Automation

- Pagination is a system where web servers will not give you all data on a single page
 - Used to reduce the load:
Loading thousands of entries is expensive
 - Even APIs do paginate!



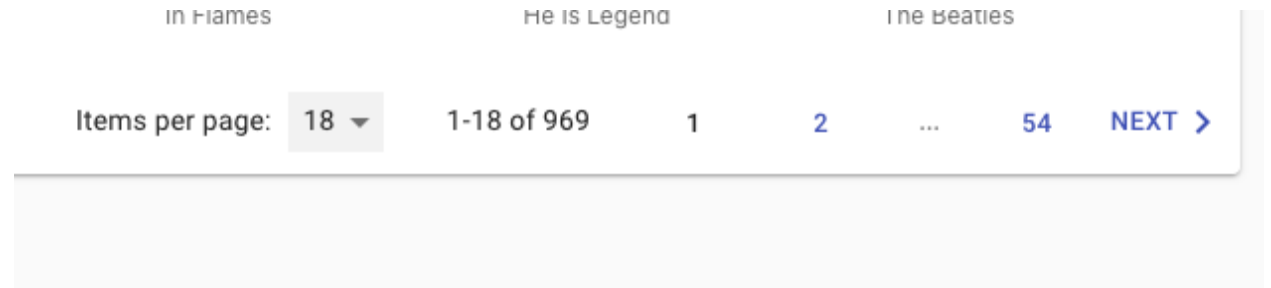
Automation

- There are several ways to perform pagination
 - Simple: The server determines the size of pages



Automation

- There are several ways to perform pagination
 - Simple: The server determines the size of pages
 - Complex: You determine the size of pages (within limits)



Corresponding URL: example.com/data?page=1&perPage=18

Automation

- Pagination is performed using URL parameters

- Simple: Parameters are provided as-is in the URL
- Complex: The website uses “pretty URLs”

`https://example.com/data/1`

`https://example.com/data/page/1`

`https://example.com/data?page=1`

Automation

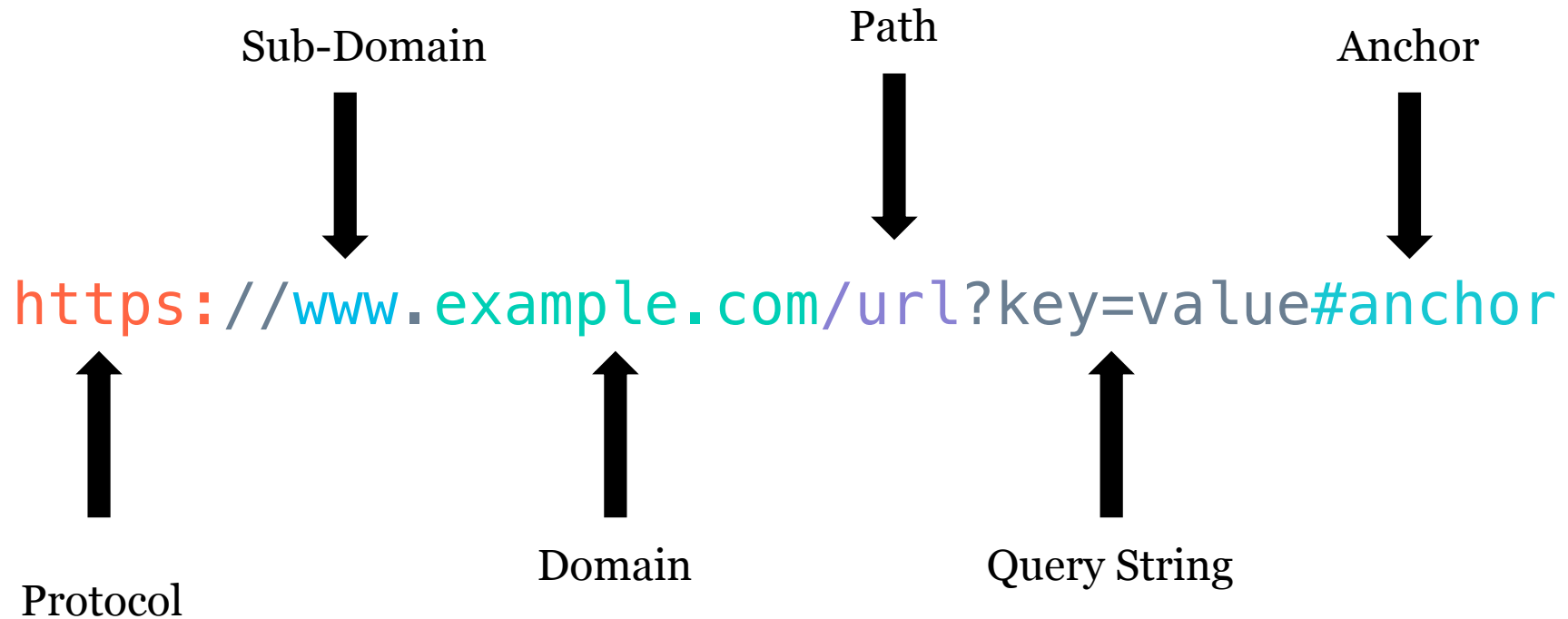
`https://data.srv.example.com/app/#/data?displayedFilters=%7B%22artist_id%22%3Atrue%7D&filter=%7B%22artist_id%22%3A%222mPBVoI5pmYsFVaqmG3Eb8%22%7D&order=ASC&page=1&perPage=18&sort=name`

Note: Websites sometimes do weird things with their URLs.
This is an (almost verbatim) existing URL that I have seen.

Automation

- Websites oftentimes use URL parameters to store search queries, pagination, or filters
- This is known as a query string, or “GET-parameters”
- **→ We can use these to our advantage and scrape several pages automatically!**

Automation



For-Loops

- When you know in advance how many pages there are, you can quickly go through them using a for-loop
 - Syntax: “For (each) element X in list Y, do...”

```
base = "https://example.com/data?page="
```

```
for (i in 1:10) {  
  url <- paste0(base, i)  
  response <- GET(url)  
}
```

For-Loops

- Tip: R's “paste” functions are smart: Passing a list as an argument will create a list of URLs!

```
> paste0("https://example.com/data?page=", 1:10)
[1] "https://example.com/data?page=1" "https://example.com/data?page=2"
[3] "https://example.com/data?page=3" "https://example.com/data?page=4"
[5] "https://example.com/data?page=5" "https://example.com/data?page=6"
[7] "https://example.com/data?page=7" "https://example.com/data?page=8"
[9] "https://example.com/data?page=9" "https://example.com/data?page=10"
> paste0("https://example.com/data?category=", c("one", "two", "three"))
[1] "https://example.com/data?category=one" "https://example.com/data?category=two"
[3] "https://example.com/data?category=three"
```

While-Loops

- Sometimes we don't know in advance how many pages there are
- In these cases, you can use a while-loop
 - Syntax: “While condition X is true, do...”

```
page <- 1
status_code <- 200

while (status_code == 200) {
  response <- GET(paste0(base, page))
  status_code <- response$status_code
  page <- page + 1
}
```

Functions

- At a certain point of complexity, it makes sense to make your code modular
- You do this with functions

```
function makeURL (page) {  
  base <- "example.com?page="   
  url <- paste0(base, page)  
  return(url)  
}
```

```
response <- GET(makeURL(1))
```

Functions

- This becomes especially powerful when combined with loops

```
function makeURL (page) {  
  base <- "example.com?page="   
  url <- paste0(base, page)  
  return(url)  
}
```

```
for (i in 1:10) {  
  response <- GET(makeURL(i))  
}
```

URL Helpers

Dealing with URLs by modifying them using `paste` can be cumbersome.

`https://www.example.com/url?key=value#anchor`

URL Helpers

Dealing with URLs by modifying them using `paste` can be cumbersome.
But of course, they have a well-defined structure!

`https://www.example.com/url?key=value#anchor`

URL Helpers

Tip: Use `parse_url` and `build_url` instead!

```
# Deconstruct a URL into its constituent parts
test_url <- parse_url("https://www.example.com/blog/?page=3&query=hello")
# Replace the "blog" part with "archive"
test_url$path = '/archive'
# Set the page parameter to number 5
test_url$query$page = 5
# Change the (search) query
test_url$query$query = "Hello there!"
# Take these parts and make a string out of it
build_url(test_url)
# Prints: "https://www.example.com/archive?page=5&query=Hello%20there%21"
```

▼ test_url	[scheme = "https", hostname = "www.example.com", port = NULL, path = "/blog/", query = [page = "3", query = "hello"], params = NULL, fragment = NULL, username = NULL, password = NULL]
scheme	"https"
hostname	"www.example.com"
port	NULL
path	"blog/"
> query	[page = "3", query = "hello"]
params	NULL
fragment	NULL
username	NULL
password	NULL

Any questions?

Thank you!